



Security Assessment

Final Report



Aave Wormhole a.DI

Adapter

June 2026

Prepared for AAVE Labs

Table of Contents

Project Summary.....	3
Project Scope.....	3
Project Overview.....	3
Protocol Overview.....	3
Assessment Methodology.....	5
Threat and Security Overview.....	6
Findings Summary.....	9
Severity Matrix.....	9
Detailed Findings.....	10
Informational Severity Issues.....	11
I-01: Unsupported Wormhole chain ID defaults to originChainId = 0.....	11
Disclaimer.....	12
About Certora.....	12

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Aave Wormhole a.DI Adapter	https://github.com/ave-dao/aave-delivery-infrastructure	b41ad47	EVM

Project Overview

This document describes the security review of **Aave Wormhole a.DI Adapter**. The work was undertaken from **June 24th, 2026** to **June 26th, 2026**.

The following contract list is included in our scope:

```
src/contracts/adapters/wormhole/WormholeExecutorAdapter.sol
src/contracts/adapters/wormhole/IWormholeExecutorAdapter.sol
src/contracts/adapters/wormhole/interfaces/ICoreBridge.sol
src/contracts/adapters/wormhole/interfaces/IExecutorQuoterRouter.sol
src/contracts/adapters/wormhole/interfaces/IVaaV1Receiver.sol
src/contracts/adapters/wormhole/libs/WormholeExecutorEncoding.sol
src/contracts/adapters/BaseAdapter.sol
src/contracts/libs/Errors.sol
```

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

This review covers the changes introduced in the pull request adding the Wormhole Executor bridge adapter to the Aave Delivery Infrastructure (a.DI). a.DI is the cross-chain messaging layer used by Aave governance. It sends the same message in parallel through several independent

bridge adapters, and the destination `CrossChainController` only acts on it once a configurable number of distinct adapters have each delivered the identical message. This requires an attacker to compromise multiple independent bridges rather than one. This PR adds Wormhole, via its Executor framework, as one such bridge for networks where Wormhole's legacy standard relayer is unavailable (e.g., Monad).

The adapter has two paths:

- Send (`forwardMessage` -> `_publishAndRequest`): publishes the raw a.DI envelope to the Wormhole Core bridge and separately pays an on-chain quote (`quoteExecution` / `requestExecution`) to a relay provider to deliver it, drawing both the Core message fee and the executor fee from the `CrossChainController`'s balance via `delegatecall`.
- Receive (`executeVAAv1`): delivery is permissionless - any caller may submit a guardian-signed VAA. The adapter verifies it on-chain via `parseAndVerifyVM`, checks the emitter against the configured trusted remotes, and forwards the payload to the `CrossChainController`. Replay protection and destination-chain binding are intentionally delegated to the controller.

The surrounding a.DI components - `CrossChainForwarder/CrossChainReceiver`, adapter registration, and the confirmation-threshold logic the adapter relies on for replay and destination protection - are outside the scope of this PR and are treated as relied-upon infrastructure.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

Scope and Trust Assumptions

The review focused on the correctness and security of `WormholeExecutorAdapter.sol` and its integration with the Wormhole Executor framework, including the on-chain quoting and execution request flow, VAA verification, trusted remote enforcement, and interaction with the `CrossChainController`.

The following are treated as trust assumptions – they were not verified in depth and must hold for the adapter to operate safely:

- Wormhole Guardian network integrity: the Guardian set (19 nodes) is honest and a quorum of 13 will not collude to forge or censor VAAs. The adapter's entire security model rests on this assumption.
- Wormhole Core Bridge correctness: `parseAndVerifyVM` correctly verifies Guardian signatures and accurately reconstructs the emitter address from the original `publishMessage` event.
- Executor relay provider liveness: a relay provider monitors `RequestForExecution` events and delivers VAAs to the destination. If no provider acts, messages can still be delivered manually by anyone with the VAA bytes, but require active intervention.
- Correct deployment configuration: trusted remote addresses and the refund address are set correctly at deployment. Both are immutable post-deployment with no recovery path.

Security Considerations and Threat Analysis

Message Authenticity. The core security property is that only messages genuinely published by a trusted source CCC can be accepted on the destination chain. Threats include:

- Forging a VAA with a trusted emitter address without a corresponding `publishMessage` call – prevented by the Guardian quorum signature requirement.
- Calling `executeVAAv1` with a VAA from an untrusted emitter – rejected by the `_trustedRemotes` check.

- Calling `executeVAAv1` with a VAA from an unsupported chain whose `nativeToInfraChainId` maps to 0 – rejected unconditionally by the CCC's `envelope.originChainId` check regardless of adapter configuration.

Message Delivery Integrity. Messages must be delivered exactly as published, without modification or substitution. Threats include:

- Substituting the VAA payload on delivery – the VAA payload is the exact bytes passed to `publishMessage` by the CCC; any modification invalidates the Guardian signatures.
- Replaying the same VAA to trigger double execution – the CCC deduplicates per (`envelopeId`, `adapter`), making double delivery a no-op.

Relay Economics. The fee model must correctly price destination gas to ensure messages are reliably delivered. Threats include:

- Underpaying the executor fee causing the relay provider to ignore the request – the on-chain quoter prices based on `dstGasPrice` and `gasLimit`. If the quoted fee is stale at execution time the transaction may revert.
- Providing insufficient `executionGasLimit` causing `executeVAAv1` to run out of gas on the destination – the relay provider has already been paid and has no obligation to retry. Manual redelivery via a direct `executeVAAv1` call is possible.

Access Control and Deployment Configuration. The adapter has no admin functions post-deployment. All security-critical parameters (`WORMHOLE_CORE`, `EXECUTOR_QUOTER_ROUTER`, `QUOTER`, trusted remotes) are set immutably in the constructor. Threats include:

- Deploying with incorrect trusted remote addresses – the adapter would reject legitimate messages or potentially accept messages from wrong sources.

Adversarial Incentives

Attacks are costly. We consider the following incentive categories:

1. Griefing: cause a message to never be delivered by underpaying the executor fee, forcing manual intervention by the sender.

2. Message injection: insert a malicious governance message into the a.DI pipeline – not achievable without compromising the Guardian quorum.
3. Replay: trigger the same governance action twice – prevented by CCC deduplication.
4. Censorship: prevent a legitimate message from being delivered – the permissionless executeVAAv1 ensures any party can act as the final delivery mechanism, making censorship resistant.
5. Fee drain: cause the CCC to overpay for message delivery – bounded by the on-chain quote at time of execution.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	-	-	-
Low	-	-	-
Informational	1	1	-
Total	1	1	-

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
I-01	Unsupported Wormhole chain ID defaults to originChainId = 0	Informational	Acknowledged

Informational Severity Issues

I-01: Unsupported Wormhole chain ID defaults to originChainId = 0

Files:

src/contracts/adapters/wormhole/WormholeExecutorAdapter.sol

Description:

In `executeVAAv1`, if a VAA arrives from a Wormhole chain not present in `nativeToInfraChainId`, the function returns `0`.

The trusted remote check then becomes `_trustedRemotes[0] == srcAddress`. If `_trustedRemotes[0]` was ever set at deployment, a message from any unsupported chain whose emitter matches that address would pass the adapter-level check. `_trustedRemotes` is immutable post-deployment (no setter exists). The same pattern exists across all adapters in the repo.

In practice this is not exploitable. Even in the worst-case misconfiguration (adapter approved for chain `0` in CCC, confirmations configured for chain `0`), point #2 is unconditional:

1. `onlyApprovedBridges(originChainId)` – adapter must be registered for chain `0` in CCC.
2. `envelope.originChainId == originChainId` – this is unconditional, the envelope is crafted by the source CCC with the real chain ID, so `real_chain_id == 0` always fails with `CHAIN_ID_MISMATCH` regardless of any configuration.
3. `requiredConfirmations > 0` – if chain `0` has no confirmations configured, message is silently dropped – configuration-dependent

Recommendations:

Add an explicit `require(originChainId != 0, Errors.ORIGIN_CHAIN_ID_NOT_SUPPORTED)` check after the `nativeToInfraChainId` call to make the intent clear and remove reliance on deployment config correctness.

Customer Response:

Acknowledged, given the impact is not exploitable.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.